2026 진학반 - 11주차

AI 에이전트 · 하네스 · MCP - 내 도구를 AI에게 쥐여 주기

모든 수업 실습 자료는 수업용 GitHub 저장소(ccube-coding)에서 확인할 수 있다.

오늘의 목표

목표	확인 방법
AI 에이전트를 「모델 + 도구 + 루프」로 설명한다	실행 로그의 생각·도구 요청·결과 단계를 가리키며 반복 한 바퀴를 말로 설명함
에이전트에 도구를 하나 더 붙여 여러 단계 질문을 풀게 한다	「중간고사 첫날 급식?」 질문에 학사일정 → 급식 순서로 도구 2개가 호출됨
하네스가 모델 바깥에서 하는 일을 안다	내가 만든 하네스와 Claude Code의 차이를 비교표로 정리함
MCP로 내 Python 함수를 Claude Code의 도구로 연결한다	Claude Code의 `/mcp`에 내 서버가 보이고, 내 도구로 급식을 조회함

오늘 완성할 프로그램

김반장 에이전트 + school MCP 서버. 10주차에 만든 급식 조회 코드를 ① 내가 만든 에이전트와 ② Claude Code, 두 곳에서 도구로 쓴다.

기능	최소 내용
에이전트 루프 관찰	`step5_agent.py`를 실행해 「생각 → 도구 요청 → 결과」 반복을 로그로 확인
도구 추가 (학사일정)	Claude Code로 학사일정 도구를 붙여, 도구 2개가 필요한 질문에 답하게 함
MCP 서버 연결	`school_mcp.py`를 Claude Code에 연결해 「내일 급식 뭐야?」에 답하게 함

폴더 구조

```
school_agent/
├── neis.py              (나이스 API 모듈 - 10주차 + 학사일정 함수 get_schedule)
├── step5_agent.py       (실습 1 - 내가 만든 에이전트 / 하네스)
├── school_mcp.py        (실습 2 - Claude Code에 꽂는 MCP 서버)
├── requirements.txt     (openai, requests, python-dotenv, mcp)
└── notes/
    ├── agent-log.md     (관찰 기록 · 하네스 비교표)
    └── why-agent.md     (회고)
```

성공 기준

- 실행 로그를 보며 에이전트가 「생각 → 도구 → 결과」를 반복한다는 것을 설명할 수 있다.
- 「중간고사 첫날 급식」처럼 도구 2개가 필요한 질문에, 순서를 알려주지 않아도 에이전트가 스스로 순서대로 조회한다.
- Claude Code의 `/mcp` 화면에 내가 만든 `school` 서버가 보이고, Claude Code가 내 도구로 조회한 급식이 나이스 원문과 같다.
- 에이전트 · 하네스 · MCP를 각각 한 문장으로 설명할 수 있다.

1. 오늘 배우는 개념

1.1. 지난주에 우리는 이미 에이전트를 만들었다

10주차 챗봇에게 「9월 17일 급식 알려줘」라고 물으면 이런 일이 일어났다.

1. AI가 「get_school_meal을 실행해 줘」라고 **요청만** 한다.
2. 우리 코드가 나이스에서 **실제로 조회**한다.
3. 조회 결과를 AI에게 다시 보여 준다.
4. AI가 결과를 보고 답한다. 더 필요하면 1번으로 돌아간다.

이렇게 **스스로 판단해서 도구를 쓰고, 결과를 보고, 답이 나올 때까지 반복하는 AI**를 AI 에이전트라고 한다.

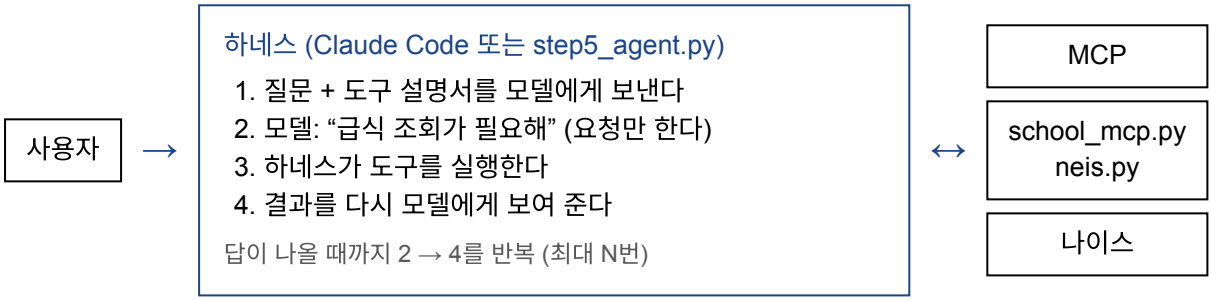
1.2. 오늘 배우는 3개 키워드

키워드	한 줄 정의
AI 에이전트(Agent)	목표를 받으면 「생각 → 도구 사용 → 결과 확인」을 답이 나올 때까지 반복하는 AI. 공식으로 쓰면 **모델 + 도구 + 루프**.
하네스(Harness)	모델 바깥에서 그 반복을 돌려 주는 프로그램. 도구를 실제로 실행하고, 반복 횟수를 제한하고, 위험한 일 전에 허락을 받고, 대화를 기억한다. `step5_agent.py`는 작은 하네스, Claude Code는 아주 큰 하네스다.
MCP(Model Context Protocol)	도구를 어느 하네스에나 꽂을 수 있게 맞춘 표준 규격. 「AI용 USB-C」라고 생각하면 된다. 한 번 만들면 Claude Code, Claude 데스크톱 등 MCP를 지원하는 곳 어디서나 쓸 수 있다.

1.3. 비유로 이해하기

부품	비유
모델	아주 똑똑하지만 **손이 없는 두뇌**
도구	**손** (급식 조회, 일정 조회, 파일 쓰기…)
하네스	두뇌와 손을 연결하고 「이건 해도 돼?」「열 번 넘게 하면 멈춰」를 관리하는 **몸**
MCP	어느 몸에나 끼울 수 있는 **표준 규격의 손**

1.4. 그림으로 보기



AI는 직접 아무것도 실행하지 못한다. 요청은 AI가, 실행은 하네스(우리 코드 또는 Claude Code)가 한다. Claude Code가 내 파일을 고칠 때도 마찬가지다.

2. 실습 1 — 에이전트 루프 눈으로 보기

2.1. 환경 준비

선생님이 준 `week11` 폴더를 `school_agent`라는 이름으로 복사한다. 터미널에서 Python 버전을 확인하고 패키지를 설치한다.

Shell
```
python --version
python -m pip install -r requirements.txt
```

⚠️ Python **3.10 이상**이어야 합니다(MCP 때문). 3.9 이하라면 선생님께 알려 주세요.

`OPENAI_API_KEY` 환경 변수는 10주차와 똑같이 등록한다. **코드에 키를 적지 않는다.**

2.2. 에이전트 코드의 핵심

`step5_agent.py`에서 가장 중요한 부분은 이 반복문이다.

Python
```
def run_agent(question: str) -> str:
    work = [{"role": "user", "content": question}]
    # 최대 5번까지만 반복한다 (하네스의 안전장치)
    for step in range(1, MAX_STEPS + 1):
        print(f"\n🧠 [{step}단계] 모델에게 묻는 중...")
        response = client.responses.create(
            model=MODEL, instructions=INSTRUCTIONS,
            input=work, tools=TOOLS, temperature=0.2,
        )
        calls = [item for item in response.output if item.type ==
"function_call"]
        # 도구를 부르지 않았다 = 최종 답을 냈다
        if not calls:
            return response.output_text

        work += response.output
        for call in calls:
            print(f"🔧 도구 요청: {call.name}({call.arguments})")
            # 실행은 우리 코드가 한다
            output = run_tool(call.name, call.arguments)
            print(f"👀 결과: {output[:200]}")
            work.append({"type": "function_call_output",
                         "call_id": call.call_id, "output": output})
    return f"{MAX_STEPS}단계 안에 답을 못 찾았어요. 질문을 더 구체적으로 해 주세요."
```

로그 기호	뜻
🧠	모델이 생각하는 단계 (OpenAI 호출 1번)
🔧	모델이 「이 도구를 이 값으로 실행해 줘」라고 요청
👀	우리 코드가 실행한 결과를 모델에게 보여 줌
✅	모델이 더 이상 도구가 필요 없다고 판단하고 낸 최종 답

2.3. 실행하고 관찰하기

```Shell
python step5_agent.py
나: 서울과학고등학교 9월 29일 급식 뭐야?
```

로그 예시:

```None
🧠 [1단계] 모델에게 묻는 중...
🔧 도구 요청: get_school_meal({"school_name":"서울과학고등학
교","date":"20260929"})
👀 결과: {"school": "서울과학고등학교", "date": "20260929", "menu": "*율무밥
..."}

🧠 [2단계] 모델에게 묻는 중...

✅ 답: 9월 29일 서울과학고 점심은 율무밥, 육개장, ...
```

이번에는 **도구가 없는 질문**도 해 본다.

```None
나: 서울과학고 다음 주 학사일정 알려줘
```

학사일정 도구가 아직 없으므로 「확인 필요」나 「조회할 수 없다」고 답해야 정상이다. 일정을 지어냈다면 `notes/agent-log.md`에 기록한다. 우리 학교 이름으로도 해 본다. 단, 주말·방학에는 급식 정보가 없다.

2.4. Claude Code로 도구 하나 더 붙이기

`step5_agent.py`에는 `TODO(실습 1-2)`라고 적힌 곳이 3군데 있다. Claude Code에게 아래처럼 부탁한다.

```
None
step5_agent.py의 TODO(실습 1-2) 3곳을 채워서 학사일정 도구 get_school_schedule을 추가해 줘.
- neis.py의 get_schedule(office_code, school_code, from_date, to_date)를 사용해
- 인자는 school_name, from_date, to_date (날짜는 YYYYMMDD)
- get_school_meal과 같은 모양으로 만들고, 다른 코드는 건드리지 마
```

추가가 끝나면 **도구 두 개가 필요한 질문**을 하고, 로그에서 학사일정 → 급식 순서로 호출되는지 확인한다.

```
None
나: 서울과학고 중간고사 첫날 급식이 뭐야? 10월 5일부터 2주 안에 있어
```

생각해 보기: 「학사일정을 먼저 보고, 그 날짜로 급식을 조회하라」는 순서를 우리가 코드에 적었나? 적지 않았다. 모델이 도구 설명서만 보고 스스로 계획을 세웠다. 이것이 「도구 한 번 부르기」와 「에이전트」의 차이다.

2.5. 검증 체크리스트

- ☐ 급식 질문에서 생각 → 도구 요청 → 결과 → 최종 답 로그를 확인했다.
- ☐ 도구를 붙이기 전, 학사일정 질문에 일정을 지어내지 않는 것을 확인했다.
- ☐ 도구를 붙인 후, 「중간고사 첫날 급식」 질문에 도구 2개가 순서대로 호출되는 것을 확인했다.
- ☐ `MAX_STEPS = 5`를 `1`로 바꿔 실행해 보고 무슨 일이 생기는지 확인했다.

3. 실습 2 — 내 코드를 MCP 서버로 만들기

3.1. MCP 서버 코드 읽기

```python
from mcp.server import MCPServer

import neis

mcp = MCPServer("school")


@mcp.tool()
def school_meal(school_name: str, date: str) -> str:
    """나이스(NEIS) 공식 데이터에서 학교의 급식(중식) 메뉴를 조회한다. date는 YYYYMMDD."""
    school = neis.get_school_code(school_name)
    if school is None:
        return "학교를 찾지 못했습니다. 정확한 학교 이름(예: 서울과학고등학교)이 필요합니다."
    meal = neis.get_meal(school[0], school[1], date)
    if meal is None:
        return f"{school[2]}의 {date} 급식 정보가 없습니다. (주말·방학·휴업일일 수 있음)"
    return f"[{school[2]} {date} {meal['meal_name']}]\n{meal['menu']}\n칼로리: {meal['calorie']}"


if __name__ == "__main__":
    mcp.run()
```

`step5_agent.py`와 비교해 보자.

구분	step5_agent.py	school_mcp.py
도구 설명서	`TOOLS`를 손으로 작성	함수 이름·타입·설명(docstring)이 자동으로 설명서가 됨
반복(루프)	직접 작성	없음. Claude Code(하네스)가 해 준다
모델 호출·API 키	있음	없음
쓸 수 있는 곳	이 파일 안에서만	MCP를 지원하는 모든 하네스

MCP 서버에는 **도구만** 있다. 도구와 하네스가 분리되었다는 것이 핵심이다.

3.2. 혼자 실행해 보기

```
Shell
python school_mcp.py
```

아무것도 안 나오고 멈춰 있으면 정상이다. Claude Code가 말을 걸어 주기를 기다리는 중이다. `Ctrl + C`로 끈다.

⚠️ MCP 서버 파일에는 `print()`를 넣지 마세요. 화면 출력 자체가 Claude Code와 대화하는 통로라서, print를 넣으면 대화가 깨집니다.

3.3. Claude Code에 연결하기

① **python 위치(절대경로) 찾기.** `mcp`를 설치한 바로 그 python이어야 한다.

컴퓨터	명령	결과 예시
Mac	`which python`	`/Users/me/school_agent/.venv/bin/python`
Windows	`where python`	`C:\Users\me\AppData\Local\Programs\Python\Python312\python.exe`

② **등록하기.** `school_agent` 폴더에서 실행한다. 경로는 내 컴퓨터에 맞게 바꾼다.

```
Shell
# Mac
claude mcp add school -- \
  /Users/me/school_agent/.venv/bin/python \
  /Users/me/school_agent/school_mcp.py

# Windows
claude mcp add school -- C:\Users\me\...\python.exe
C:\Users\me\school_agent\school_mcp.py
```

- `school`: 내가 붙이는 서버 이름
- `--` 뒤: 서버를 실행하는 명령 (python 경로 + 파일 경로, 둘 다 절대경로)

③ **연결 확인.**

```
Shell
claude mcp list
```

`school`이 Connected로 보이면 성공이다. `school_agent` 폴더에서 Claude Code를 켜고 `/mcp`를 입력해 `school` 서버와 `school_meal` 도구가 보이는지 확인한다.

3.4. Claude Code에게 물어보기

```
None
서울과학고등학교 내일 급식 뭐야? school 도구로 조회해 줘.
```

관찰할 것:

1. Claude Code가 도구를 쓰기 전에 **허락을 묻는가?**
2. 답과 함께 **도구 호출 기록**이 보이는가? `step5`의 도구 요청 로그와 비교해 본다.
3. 이어서 「그럼 모레는?」이라고 물으면 학교 이름을 **기억하는가?**

3.5. 도전 — MCP 서버에도 학사일정 도구 추가

`school_mcp.py`의 `TODO`를 Claude Code에게 채워 달라고 한다. 서버 코드가 바뀌면 Claude Code를 다시 시작해야 반영된다. `/mcp`에서 도구 2개를 확인한 뒤 「중간고사 첫날 급식」을 물어본다.

3.6. 검증 체크리스트

- [ ] `claude mcp list`에서 `school`이 Connected로 보이는 것을 확인했다.
- [ ] Claude Code `/mcp`에서 `school_meal` 도구가 보이는 것을 확인했다.
- [ ] Claude Code가 내 도구로 조회한 급식이 나이스 원문과 같은 것을 확인했다.
- [ ] (도전) 도구 2개가 보이고, 두 도구를 순서대로 쓰는 질문에 답하는 것을 확인했다.

4. 하네스 비교 — 같은 도구, 다른 하네스

실습하며 관찰한 내용으로 `notes/agent-log.md`의 비교표를 채운다.

구분	내 하네스 (step5_agent.py)	Claude Code
도구 설명서는 누가 썼나		
도구 실행 전 허락을 받나		
최대 몇 번 반복하나		
이전 대화를 기억하나		

같은 모델, 같은 도구라도 **하네스가 다르면 안전성과 편리함이 달라진다.** AI 회사들이 모델만큼 하네스를 열심히 만드는 이유다.

5. 회고와 숙제

5.1. 회고 — notes/why-agent.md

- 에이전트 / 하네스 / MCP를 각각 한 문장으로 설명하기
- 내 캡스톤에 AI를 붙인다면 어떤 도구를 줄까? AI에게 맡기지 않을 것은?

5.2. 숙제 — 캡스톤 주제 준비 (필수)

다음 주 시작하자마자 캡스톤 주제를 확정한다. 아래를 해 온다.

- ☐ 「캡스톤 주제 선정」 자료의 몸풀기 질문지 A~E에 답을 적고, 마음이 가는 3개에 ★를 표시했다.
- ☐ 그중 1개로 「나는 [누구]의 [어떤 불편]을 [무엇]으로 돕는다」 한 문장을 썼다.
- ☐ (선택) 그 주제에 쓸 수 있는 공공데이터·API를 하나 찾았다.

트러블슈팅

증상	원인 및 해결 방법
`No module named 'mcp'`	`python -m pip install -r requirements.txt`를 실행할 때 쓰는 그 python으로 다시 설치
`No module named 'mcp.server.fastmcp'`	인터넷·AI가 준 옛날(1.x) 예제. `from mcp.server import MCPServer`, `MCPServer("school")`로 바꾸기
`claude mcp list`에서 연결 실패	python 경로·파일 경로가 절대경로인지, 그 python에 mcp가 설치됐는지 확인. `claude mcp remove school` 후 다시 add
Claude Code가 내 도구를 안 씀	질문 끝에 「school 도구로 조회해 줘」 붙이기
서버 코드를 고쳤는데 그대로임	Claude Code 껐다 켜기
`python school_mcp.py`가 멈춰 있음	정상. `Ctrl + C`로 종료
`RateLimitError` / 429	사용 한도 초과. 선생님께 알리기 (실습 2는 OpenAI 키 없이도 된다)
학사일정이 5개까지만 나옴	나이스 샘플 모드 제한. 기간을 1~2주로 좁히기

완성 확인 목록

- ☐ `step5_agent.py` 실행 로그를 보고 에이전트 루프 한 바퀴를 설명할 수 있다.
- ☐ 학사일정 도구를 붙여 도구 2개가 필요한 질문에 답하게 했다.
- ☐ `school_mcp.py`를 Claude Code에 연결해 급식을 조회했다.
- ☐ 에이전트 · 하네스 · MCP를 각각 한 문장으로 설명할 수 있다.